LEADERS UNIVERSITY / LICENCE BI

Python, ETL et entrepôts de données

Atlas Papeterie · Licence BI · 28 h · 70 % de pratique

28 h	70 %	0
PARCOURS GUIDÉ	PRATIQUE	PRÉREQUIS IT

ATLAS PAPETERIE

Fichiers de ventes → ETL → Entrepôt → Rapport

02 / LE COURS ET LES MISSIONS

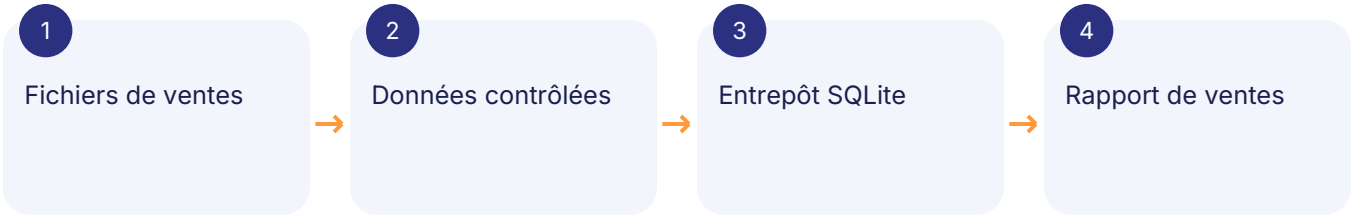
Youssef Douma · Leaders University

Édition pédagogique · 26 octobre 2026 · données fictives

BI / NOTRE POINT DE DÉPART

Une question, un projet, un résultat

Atlas Papeterie possède deux boutiques fictives : Nabeul (NBE) et Tunis (TUN). Les ventes arrivent dans des fichiers. La responsable veut savoir quels produits suivre avant de préparer les prochains achats. Votre mission : rendre ces fichiers fiables, les ranger dans un entrepôt de données et produire un rapport lisible.



Ce que vous construirez

- Un script Python que vous savez lancer, modifier et expliquer.
- Un journal des anomalies : chaque ligne écartée conserve une raison.
- Un entrepôt en étoile : trois dimensions et une table de faits.
- Un rapport HTML local, consultable dans votre navigateur sans compte.

Vous partez de zéro. Savoir déjà programmer, utiliser SQL ou installer un serveur n'est pas nécessaire. Nous apprendrons les gestes au moment où ils deviennent utiles. Le nom du module est « Langage avancé » ; le parcours commence volontairement par les fondations.

Une limite à garder en tête

Ces ventes ne décrivent ni les stocks, ni les coûts, ni la marge. Elles permettent d'observer une activité passée, pas de prédire une demande ou de décider seules des achats.

LE PARCOURS / 10 MISSIONS

Votre itinéraire sur 28 heures

Mission	Ce que vous produisez	Durée
01 · Premiers gestes	Un calcul expliqué et enregistré	3 h
02 · Lire	Un aperçu et le sens des colonnes	3 h
03 · Nettoyer	Des ventes retenues et des rejets justifiés	3 h
04 · Transformer	Des montants en millimes contrôlés	3 h
05 · SQL	Une réponse filtrée et une somme	3 h
06 · Modéliser	Un schéma en étoile lisible	3 h
07 · Charger	Un entrepôt peuplé et contrôlé	3 h
08 · Incrémenter	Un ajout rejouable sans doublon	3 h
09 · Expliquer	Un rapport et une conclusion limitée	3 h
10 · Défi	Une réparation et une démonstration	1 h

Répartition planifiée : 504 minutes de théorie (8 h 24) et 1 176 minutes de pratique (19 h 36), soit 30 % / 70 %. La pratique comprend lecture active, prédiction, saisie, essais, dépannage et explication. Une exécution automatique de quelques secondes ne représente pas le travail d'apprentissage.

Votre rythme

Les durées sont des estimations de préparation, pas des temps mesurés auprès de débutants. Terminer le socle avec une explication vaut mieux que parcourir toutes les extensions sans comprendre.

LA MÉTHODE / PRÉDIRE → ESSAYER → EXPLIQUER

Comment apprendre dans cet atelier

1. Observer le cas → 2. Prédire le résultat → 3. Changer une chose → 4. Vérifier et expliquer

Le script de référence fonctionne déjà. Il vous permet d'observer un résultat avant de modifier une petite partie. Vous ne devez pas recopier tout un programme ni comprendre chaque ligne du moteur technique dès la première séance.

- 1. Ouvrir le fichier indiqué dans Thonny et lire ses commentaires.
- 2. Exécuter le cas de référence avec F5 ; conserver le résultat attendu.
- 3. Écrire une prédiction dans votre journal avant la variation.
- 4. Modifier un seul paramètre, enregistrer avec Ctrl+S, puis relancer.
- 5. Comparer avant/après et expliquer la cause en une ou deux phrases.
- 6. Restaurer le paramètre initial avant de passer à la mission suivante.

Votre journal d'apprentissage

Dans le dossier du cours, créer journal_apprentissage.txt avec le Bloc-notes. Pour chaque mission, écrire : fichier utilisé ; prédiction ; résultat observé ; explication ; difficulté et réparation éventuelle. Aucun identifiant ou mot de passe n'est demandé.

Si vous êtes bloqué

Relisez la dernière ligne rouge, le nom du fichier ouvert et le paramètre modifié. Essayez une hypothèse simple. Demandez de l'aide avec ces trois informations ; ne recommencez pas toute l'installation.

REPÈRES / PYTHON, SQL ET SQLITE

Les outils, sans détour technique

Outil	Son rôle dans notre projet
Python	Lire les fichiers, appliquer les règles et automatiser les étapes.
Thonny	Écrire et lancer les programmes Python ; voir les messages.
SQL	Poser des questions à des tables et calculer des regroupements.
SQLite	Conserver nos tables dans un fichier, sans serveur à installer.
Navigateur	Ouvrir le rapport HTML produit sur votre ordinateur.

Python a été créé par Guido van Rossum ; sa documentation décrit un langage utilisé pour automatiser des tâches et construire des applications. SQL sert à manipuler des données tabulaires. SQLite est un moteur SQL intégré : dans notre atelier, la base tient dans un fichier atlas.sqlite. Ce format convient à notre petite démonstration locale ; il ne résume pas l'architecture d'un entrepôt d'entreprise.

Nous utilisons la bibliothèque standard de Python : csv pour les fichiers, pathlib pour les chemins, decimal pour lire précisément les prix, sqlite3 pour la base et html pour le rapport. Aucun paquet additionnel n'est requis dans le socle.

Sources et installation

Les liens officiels et le parcours Windows sont dans 01_demarrer_windows.pdf. Les repères de dépannage sont regroupés dans 03_reperes_et_depannage.pdf. Commencez par le guide Windows avant la mission 01.

LE CONTRAT / UNE LIGNE N'EST PAS UN CLIENT

Connaître les données avant de calculer

Champ	Sens	Exemple
sale_id	Identifiant d'une vente élémentaire	S001
date	Date de la vente : année-mois-jour	2026-01-05
shop_id	Code de la boutique	NBE
product_id	Code du produit	P01
quantity	Nombre d'articles vendus	2
unit_price_tnd	Prix d'un article, en TND	4.500

Une ligne valide décrit un produit vendu dans une boutique à une date. sale_id identifie cette vente élémentaire. Nous ne connaissons ni le nombre de clients, ni le nombre de commandes contenant plusieurs produits. Compter les lignes ne donne donc pas ces nombres.

Un jeu fictif et volontairement petit

Le fichier initial contient 16 lignes de données, sans compter l'en-tête. Après contrôle : 12 ventes distinctes retenues, 1 doublon exact et 3 rejets. Les ventes retenues portent sur les 5, 6 et 7 janvier 2026 : 32 articles, pour 99.000 TND. Le fichier d'ajout sera introduit plus tard.

Conserver une référence

Gardez data/ intact pendant le socle. Les essais qui changent les données se font sur une copie portant un autre nom. Tous les nouveaux résultats vont dans outputs/.

MISSION 01 / COMPRENDRE LE GESTE

01 · Prendre la main, sans crainte

Nous voulons calculer le montant de la vente S001 : deux carnets à 4.500 TND chacun. Avant le programme, le calcul est familier : 2 × 4.500 = 9.000 TND. Python exécutera exactement cette opération.

Quantité	Prix d'un carnet	Montant de la vente
2 articles	4.500 TND	9.000 TND
2 articles	4 500 millimes	9 000 millimes

Un dossier contient des fichiers. Un fichier .py contient du texte que Python peut exécuter. Dans Thonny, l'éditeur est la zone où vous changez le programme ; le Shell est la zone où apparaissent les résultats et les erreurs. Modifier le Shell ne modifie pas le fichier enregistré.

- Ouvrir Thonny, puis Fichier → Ouvrir (File → Open).
- Ouvrir scripts/01_premiers_pas.py dans le dossier extrait du cours.
- Repérer QUANTITE et PRIX_MILLIMES en haut du fichier.
- Appuyer sur Ctrl+S, puis F5. Lire le résultat dans le Shell.

Socle

Vous savez ouvrir le bon fichier, lancer le cas de référence et retrouver le résultat 9.000 TND. Aucun terminal Windows n'est nécessaire.

MISSION 01 / UN PREMIER PROGRAMME

01 · Nommer, calculer, afficher

Une variable donne un nom à une valeur. Le signe = range la valeur de droite sous le nom de gauche. Le signe * multiplie. print affiche un résultat : il ne le sauvegarde pas dans un document.

Exemple autonome à ouvrir dans Thonny

```
quantite = 2
prix_millimes = 4500
montant_millimes = quantite * prix_millimes
print(f"Montant : {montant_millimes / 1000:.3f} TND")
```

Cet exemple est disponible dans examples/01_calcul.py. Ouvrez-le dans Thonny, puis F5. Les deux points suivis de .3f demandent trois chiffres après le point pour l'affichage. Le calcul monétaire, lui, reste en millimes entiers.

Écriture	Ce qu'elle signifie
4500	Un nombre entier : 4 500 millimes.
"Montant"	Un texte entre guillemets.
# commentaire	Une explication que Python n'exécute pas.

Avant d'exécuter

Si la quantité passe de 2 à 3 et que le prix ne change pas, quel montant prévoyez-vous ? Écrivez votre prédiction avant de toucher au paramètre.

MISSION 01 / PRATIQUE GUIDÉE

01 · Votre premier atelier

- Revenir à scripts/01_premiers_pas.py. Conserver PRIX_MILLIMES = 4500.
- Changer QUANTITE = 2 en QUANTITE = 3 ; Ctrl+S, puis F5.
- Noter le montant avant, votre prédiction et le résultat après.
- Restaurer QUANTITE = 2. Relancer pour retrouver la référence.
- Changer uniquement PRIX_MILLIMES en 5000 ; prévoir puis vérifier.
- Restaurer PRIX_MILLIMES = 4500 et relancer.

Travail à deux : une personne annonce une variation, l'autre prédit le résultat. Inversez les rôles. Vous devez pouvoir expliquer quelle valeur représente des articles et quelle valeur représente des millimes.

Un petit défi de compréhension

Dans l'exemple autonome, ajoutez une variable nommée nom_produit contenant le texte "Carnet" et affichez-la avec print. Cette nouvelle ligne ne doit pas modifier le montant. Gardez les guillemets autour du texte.

Extension facultative

Trouvez deux couples quantité/prix donnant le même montant de 9.000 TND. Ce n'est pas une nouvelle donnée de vente : c'est une expérience de calcul séparée.

Le temps de pratique comprend les prédictions, les essais, l'enregistrement, la discussion et le dépannage. Gardez une trace lisible ; la vitesse de frappe n'est pas l'objectif.

MISSION 01 / PREUVE ET BILAN

01 · Lire une erreur et repartir

Symptôme	Première vérification
Le montant ne change pas	Ai-je enregistré et lancé le bon onglet ?
NameError	Le nom est-il écrit exactement pareil, majuscules comprises ?
SyntaxError	Ai-je effacé un guillemet ou une parenthèse ?
Le fichier semble absent	Ai-je extrait le ZIP avant de travailler ?

Pour tester une erreur sans perdre votre travail, utilisez examples/01_calcul.py. Changez une occurrence de quantite en quantitee. Lancez : Python ne trouve pas ce nouveau nom. Réparez l'orthographe, enregistrez et relancez. Cette erreur ne signifie pas que l'ordinateur ou Python est cassé.

Votre preuve

- Le script de mission est revenu à QUANTITE = 2 et PRIX_MILLIMES = 4500.
- Votre journal contient au moins deux prédictions comparées à leurs résultats.
- Vous expliquez oralement ou par écrit : fichier, éditeur, Shell, variable.

À ce stade, je sais...

Ouvrir et exécuter un programme, changer un paramètre, lire un résultat avec son unité et réparer une petite erreur. Prochaine étape : remplacer les valeurs saisies à la main par un fichier de ventes.

MISSION 02 / COMPRENDRE UN CSV

02 · Ouvrir la source des ventes

Un CSV est un fichier texte organisé en lignes et colonnes. Ici, le séparateur est la virgule. La première ligne donne les noms des colonnes ; les suivantes contiennent les données. Une date et un prix arrivent d'abord comme du texte : leur apparence ne garantit pas qu'ils sont valides.

sale_id	date	shop_id	product_id	quantity	unit_price_tnd
S001	2026-01-05	NBE	P01	2	4.500
S002	2026-01-05	NBE	P02	3	1.500

- Dans Thonny, ouvrir scripts/02_lire_csv.py.
- Lire le nom de la source : data/ventes_initiales.csv.
- Exécuter avec F5 ; observer le nombre de lignes et l'aperçu.
- Dans l'Explorateur Windows, ouvrir le CSV avec le Bloc-notes pour voir sa forme texte. Ne pas enregistrer de changement.

Attention avec Excel

Excel peut choisir un autre séparateur ou modifier les dates lors d'un enregistrement. Pour cet atelier, examinez d'abord les données avec le Bloc-notes. Il n'est pas nécessaire d'avoir Excel.

MISSION 02 / DU TEXTE AU TABLEAU

02 · Une liste de lignes nommées

Un dictionnaire associe un nom à une valeur. La ligne S001 peut être lue avec le nom quantity plutôt qu'avec « la cinquième case ». Une liste permet de conserver plusieurs lignes ; len compte ses éléments.

Exemple autonome à ouvrir dans Thonny

```
vente = {"sale_id": "S001", "quantity": "2"}
print(vente["sale_id"])
print(vente["quantity"])
print(type(vente["quantity"]).__name__)
```

Ouvrez examples/02_ligne.py et lancez-le. Vous obtenez S001, puis 2, puis str : quantity est encore une chaîne de caractères. Pour faire un calcul, il faudra convertir cette valeur et contrôler qu'elle convient.

Le chemin, sans dépendre du Bureau

Les scripts fournis retrouvent le dossier du cours à partir de leur propre emplacement. Ils fonctionnent même si le dossier Windows n'a pas le même nom que chez votre voisin. Gardez simplement ensemble scripts/, data/ et atelier.py.

À observer

L'aperçu montre seulement les premières lignes. Il ne prouve pas que toutes les lignes sont propres. Nous lirons le fichier entier avant de compter les anomalies.

MISSION 02 / ATELIER D'EXPLORATION

02 · Lire, compter, décrire

- Exécuter le script de référence avec LIMITE_APERCU = 3.
- Prévoir ce qui changera si LIMITE_APERCU devient 5.
- Modifier le paramètre, enregistrer et relancer.
- Vérifier que le nombre total de lignes reste 16 : seul l'aperçu change.
- Restaurer LIMITE_APERCU = 3.

À chercher dans la source	Question à se poser
S003 apparaît deux fois	Deux ventes ou une répétition du même enregistrement ?
Une quantité égale à 0	Cette ligne décrit-elle une vente positive ?
Le code produit P99	Existe-t-il dans data/produits.csv ?
Une date du 30 février	Une suite de caractères est-elle toujours une vraie date ?

Dans votre journal, rédigez une phrase pour chaque colonne, avec son unité si elle en a une. Puis choisissez une ligne et racontez-la en français : produit, boutique, date, quantité et prix unitaire. Le total d'une ligne n'est pas son prix unitaire.

Extension facultative

Dans l'exemple autonome, ajoutez la clé "shop_id" avec "NBE". Affichez-la. Que se passe-t-il si vous demandez une clé qui n'existe pas ? Réparez après l'essai.

MISSION 02 / PREUVE ET BILAN

02 · Ne pas confondre aperçu et contrôle

Le fichier source compte 16 lignes de données. Ce nombre ne signifie pas « 16 ventes utilisables ». Il décrit ce que nous avons reçu. Nous n'écartons encore rien sans raison : l'étape suivante rendra chaque décision visible.

Question	Réponse que vous devez pouvoir justifier
Que représente une ligne ?	Une vente élémentaire, pas nécessairement un client.
Pourquoi des noms de colonnes ?	Pour retrouver les valeurs sans deviner leur position.
Pourquoi regarder plusieurs lignes ?	Pour repérer formats, répétitions et valeurs suspectes.
Pourquoi conserver la source ?	Pour pouvoir refaire le traitement et comprendre un écart.

Si le fichier ne s'ouvre pas

- FileNotFoundError : vérifier que le ZIP a été entièrement extrait.
- Vérifier le nom ventes_initiales.csv et le dossier data/.
- Ne pas déplacer seulement un script sur le Bureau ; garder le dossier complet.
- Si une colonne est introuvable, vérifier l'en-tête et les virgules.

À ce stade, je sais...

Lire le sens d'un tableau, distinguer texte et nombre, compter les lignes reçues et expliquer pourquoi un aperçu n'est pas un contrôle qualité.

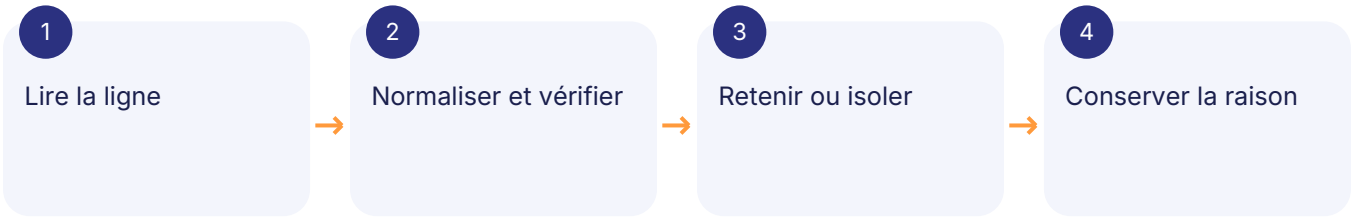
MISSION 03 / LA QUALITÉ DES DONNÉES

03 · Corriger une forme, pas inventer un fait

Nettoyer signifie appliquer des règles explicites. Enlever des espaces autour de " nbe " et écrire NBE permet de reconnaître le même code. En revanche, inventer un produit pour P99 ou remplacer une date impossible par une date devinée modifierait le sens des données.

Valeur reçue	Décision	Pourquoi
" nbe "	Normaliser en NBE	Même code après espaces et casse.
Quantité 0	Rejeter avec raison	Notre règle exige une quantité positive.
Produit P99	Rejeter avec raison	Absent du catalogue.
2026-02-30	Rejeter avec raison	Date impossible.

Un doublon exact est une répétition de toutes les valeurs normalisées d'une même vente. Conserver les deux compterait la même vente deux fois. Deux lignes ayant le même identifiant mais des valeurs différentes sont un conflit, pas un simple doublon : nous ne choisissons pas arbitrairement.



Socle

Une ligne écartée n'est pas une ligne oubliée : elle apparaît dans le suivi des rejets ou des doublons.

MISSION 03 / COMPRENDRE STRIP ET UPPER

03 · Une règle simple, puis le contrôle complet

Exemple autonome à ouvrir dans Thonny

```
code_recu = " nbe "
code_propre = code_recu.strip().upper()
print(repr(code_recu))
print(code_propre)
quantite = 0
if quantite <= 0:
    print("Rejet : quantite non positive")
```

Dans examples/03_normaliser.py, strip enlève les espaces au début et à la fin ; upper met les lettres en majuscules. if signifie « si ». Les lignes décalées sous if appartiennent à la condition : gardez leur indentation.

- Ouvrir scripts/03_nettoyer.py dans Thonny.
- Conserver SOURCE = "ventes_initiales.csv" puis lancer avec F5.
- Ouvrir outputs/03/ventes_nettoyees.csv avec le Bloc-notes.
- Ouvrir outputs/03/rejets.csv puis outputs/03/doublons.csv.
- Retrouver les identifiants I001, I002, I003 et S003 dans leurs suivis.

Résultat de référence

16 reçues = 12 retenues + 1 doublon exact + 3 rejets. L'en-tête de chaque CSV ne compte pas comme une ligne de données.

MISSION 03 / ATELIER DE CONTRÔLE

03 · Votre enquête sur trois anomalies

- Pour chaque rejet, recopier l’identifiant et expliquer la règle violée dans votre journal.
- Comparer S003 dans la source et dans le fichier propre : combien de fois doit-elle apparaître dans chaque fichier ?
- Expliquer pourquoi les espaces et la casse peuvent être corrigés sans changer la vente.
- Relancer le script sans modification : vérifier que les mêmes comptes reviennent.

Variation sur une copie

Dupliquez data/ventes_initiales.csv dans le même dossier avec le nom ventes_essai.csv. Ouvrez cette copie dans le Bloc-notes. Sur une seule ligne valide, entourez shop_id d’espaces sans changer les virgules. Enregistrez. Dans le script 03, changez SOURCE en "ventes_essai.csv". Prédisez le nombre de lignes retenues, puis relancez. Gardez l’original intact.

Restaurez SOURCE = "ventes_initiales.csv", puis exécutez de nouveau. Les fichiers outputs/ redeviennent la référence pour la prochaine mission. Votre copie d’essai peut rester dans data/ avec son nom distinct.

Extension facultative

Sur une nouvelle copie, remplacez une quantité positive par 0. Prévoyez le mouvement entre retenues et rejets avant de lancer. La somme des catégories doit toujours retrouver le nombre reçu.

MISSION 03 / PREUVE ET BILAN

03 · Chaque écart doit être expliqué

Contrôle	Attendu sur la référence
Lignes lues	16
Ventes distinctes retenues	12
Doublons exacts isolés	1
Lignes rejetées	3
Réconciliation	12 + 1 + 3 = 16

Ne regardez pas seulement « succès » dans le Shell. Un traitement peut s'exécuter sans erreur technique tout en produisant un nombre faux. Ici, nous contrôlons à la fois l'exécution, les comptes et la raison métier des exclusions.

Si vous êtes bloqué

- Une indentation est cassée : restaurer les espaces dans l'exemple, pas dans le moteur atelier.py.
- Un CSV semble avoir une seule colonne : vérifier le séparateur virgule.
- Une nouvelle erreur apparaît après l'essai : revenir à SOURCE = "ventes_initiales.csv".
- Un rejet vous surprend : relire la ligne brute et le catalogue avant de proposer une correction.

À ce stade, je sais...

Distinguer normalisation, rejet, doublon et conflit ; conserver une trace des choix ; réconcilier les lignes reçues avec toutes les issues du traitement.

MISSION 04 / TRANSFORMER LES VALEURS

04 · Un montant fiable pour chaque vente

Les données sont maintenant utilisables. Nous voulons calculer le montant de chaque vente et contrôler la somme totale. Il faut distinguer une quantité, un prix unitaire et un montant. Pour S001 : 2 articles × 4.500 TND/article = 9.000 TND.

Étape	Exemple S001	Unité
Lire le prix	"4.500"	Texte reçu
Convertir précisément	4 500	Millimes par article
Multiplier par la quantité	2 × 4 500 = 9 000	Millimes
Afficher le montant	9.000	TND

Nous stockons l'argent en millimes entiers. Un dinar vaut 1 000 millimes. Le point dans le fichier sépare les décimales ; il ne sépare pas des milliers. Le prix 4.500 signifie quatre dinars et cinq cents millimes.

Pourquoi ne pas arrondir au hasard ?

Des nombres décimaux représentés en binaire peuvent avoir une petite approximation. Decimal lit le prix depuis son texte ; l'entier en millimes garde ensuite notre précision de trois décimales.

MISSION 04 / UN EXEMPLE MONÉTAIRE

04 · Convertir, multiplier, comparer

Exemple autonome à ouvrir dans Thonny

```
from decimal import Decimal

prix_texte = "4.500"
prix_millimes = int(Decimal(prix_texte) * 1000)
quantite = 2
montant_millimes = quantite * prix_millimes
print(f"{montant_millimes / 1000:.3f} TND")
print(montant_millimes >= 9000)
```

Ouvrez examples/04_montant.py. Le résultat est 9.000 TND, puis True : le montant atteint le seuil de 9 000 millimes. True et False signifient vrai et faux. Le signe >= veut dire « supérieur ou égal » ; la vente au seuil doit donc être incluse.

- Ouvrir scripts/04_transformer.py, puis exécuter la référence.
- Observer outputs/04/ventes_transformees.csv et ses nouvelles colonnes monétaires.
- Retrouver S001 et vérifier son montant à la main.
- Contrôler les totaux : 12 ventes, 32 articles, 99.000 TND.

Contrôle utile

La somme des quantités compte des articles. Le nombre de lignes compte des ventes élémentaires. La somme des montants mesure des TND : ces trois résultats ne sont pas interchangeables.

MISSION 04 / ATELIER DE TRANSFORMATION

04 · Un seuil est une question, pas un nettoyage

Le paramètre SEUIL_MILLIMES choisit les ventes à observer dans l'analyse affichée. Il ne signifie pas que les petites ventes sont mauvaises. Elles restent valides et doivent rester présentes dans l'ensemble transformé.

- Prévoir quelles ventes atteignent 9.000 TND ; les repérer dans le fichier transformé.
- Exécuter avec SEUIL_MILLIMES = 9000 et comparer à votre liste.
- Changer le seuil à 10000. Prédire l'effet de cette hausse.
- Comparer le nombre affiché au seuil et le total complet des ventes.
- Restaurer SEUIL_MILLIMES = 9000, puis relancer.

Écrire une petite règle

Dans examples/04_montant.py, ajoutez une condition qui affiche "Vente importante" lorsque le montant est au moins 9 000 millimes. Testez avec quantité 1, 2 et 3. Avant chaque essai, prévoyez si le message doit apparaître. Ne modifiez qu'une valeur à la fois.

Extension facultative

Calculez le montant moyen par vente : montant total ÷ nombre de ventes retenues. Indiquez le dénominateur et l'unité. Ce n'est ni le panier moyen par client, ni le bénéfice de la boutique.

MISSION 04 / PREUVE ET BILAN

04 · Contrôler le calcul et son sens

Contrôle à expliquer	Référence
S001	2 × 4 500 = 9 000 millimes
Nombre de ventes	12
Quantité totale	32 articles
Montant total	99 000 millimes = 99.000 TND

Quand un résultat est inattendu, vérifiez les unités avant de modifier une formule. Multiplier un prix déjà exprimé en millimes par 1 000 une seconde fois donne un montant mille fois trop grand. Mélanger texte et nombre peut aussi provoquer une erreur ou une opération qui n'a pas le sens voulu.

Votre preuve

- Votre journal contient le calcul détaillé de S001 avec ses unités.
- Vous distinguez un contrôle qualité et un filtre d'analyse.
- Vous expliquez pourquoi une vente exactement au seuil est retenue.
- Les paramètres et les fichiers de référence ont été restaurés.

À ce stade, je sais...

Convertir des valeurs textuelles, calculer des montants monétaires précis, écrire une condition et vérifier le sens d'une somme. Prochaine étape : poser nos questions avec SQL.

MISSION 05 / DÉCOUVRIR SQL

05 · Poser une question à une table

SQL permet de décrire la réponse souhaitée. Nous voulons voir les ventes de Nabeul puis connaître leur montant total. Une table a des colonnes nommées ; SELECT choisit ce que l'on affiche, FROM indique la table et WHERE garde les lignes qui respectent une condition.

Question en français	Mot SQL utile
Quelles colonnes ?	SELECT
Dans quelle table ?	FROM
Quelles lignes ?	WHERE
Quelle somme ?	SUM
Un résultat par groupe ?	GROUP BY

La question « montant des ventes de Nabeul » a deux étapes : garder shop_id = NBE, puis additionner les montants. Filtrer sur un nom qui n'existe pas ne prouve pas l'absence d'activité dans le monde réel : cela renvoie seulement aucune ligne correspondante dans nos données.



Un nouveau langage, un même métier

Le mot-clé SQL ne change pas le sens de la quantité ou du montant. Vous devez toujours savoir ce que représente une ligne.

MISSION 05 / COMPRENDRE AVANT LE VRAI FICHIER

05 · Une requête qui tient en quelques lignes

Exemple autonome à ouvrir dans Thonny

```
import sqlite3

base = sqlite3.connect(":memory:")
base.execute("CREATE TABLE ventes (boutique TEXT, montant INT)")
base.execute("INSERT INTO ventes VALUES (?, ?)", ("NBE", 9000))
base.execute("INSERT INTO ventes VALUES (?, ?)", ("TUN", 6000))
question = "SELECT SUM(montant) FROM ventes WHERE boutique = ?"
resultat = base.execute(question, ("NBE",)).fetchone()[0]
print(f"{resultat / 1000:.3f} TND")
base.close()
```

Ouvrez examples/05_sql_simple.py puis F5. Ce petit exemple crée une table temporaire dans la mémoire, y place deux lignes et répond 9.000 TND pour NBE. Il sert à comprendre SQL ; ce n'est pas encore l'entrepôt Atlas.

Le point d'interrogation reçoit une valeur séparément de la requête. Le code ne colle pas le contenu d'un texte dans SQL. Le groupe ("NBE",) contient une valeur ; sa virgule est utile en Python.

- Ouvrir ensuite scripts/05_sql.py.
- Conserver BOUTIQUE = "NBE" et exécuter.
- Repérer la requête et les colonnes utilisées.

Différence importante

Le petit exemple ne contient que deux lignes. Le script de mission utilise les ventes de référence ; son résultat pour NBE est 43.500 TND.

MISSION 05 / ATELIER SQL

05 · Filtrer, regrouper, vérifier

- Prédire le résultat si BOUTIQUE devient "TUN".
- Changer le paramètre, enregistrer et exécuter : comparer avec 55.500 TND.
- Vérifier à la main que 43.500 + 55.500 = 99.000 TND.
- Restaurer BOUTIQUE = "NBE" et relancer.
- Dans l'exemple autonome, changer uniquement la valeur passée à la requête en "TUN" : prévoir avant d'exécuter.

Passer d'un résultat à plusieurs

Un regroupement calcule une somme pour chaque valeur de boutique. Au lieu de poser deux questions séparées, GROUP BY produit une ligne pour NBE et une ligne pour TUN. Vérifiez que la somme des groupes retrouve le montant global. Un regroupement par produit répond à une autre question, avec les mêmes ventes.

Boutique	Montant observé
NBE · Nabeul	43.500 TND
TUN · Tunis	55.500 TND
Les deux boutiques	99.000 TND

Extension facultative

Dans l'exemple autonome, ajouter une troisième ligne NBE de 4 500 millimes. Prédire la nouvelle somme, exécuter, expliquer. Cette expérience ne change pas les données Atlas.

MISSION 05 / PREUVE ET BILAN

05 · Une somme n’explique pas tout

Tunis a un montant observé plus élevé que Nabeul sur les trois mêmes dates du fichier. Cela ne suffit pas à dire que son équipe travaille mieux ou que sa rentabilité est supérieure : nous ne connaissons ni horaires, ni surface, ni coûts, ni stocks.

Si la requête échoue

- no such table : le nom de table existe-t-il dans cette base ?
- no such column : comparer le nom tapé et le nom réellement créé.
- Une apostrophe ou une parenthèse manque : relire toute la requête.
- Le résultat est vide : vérifier le code de boutique avant de modifier les données.

Votre preuve

- Vous reformulez la requête en français avant de l’exécuter.
- Vous montrez les résultats NBE et TUN et leur réconciliation au total.
- Vous expliquez pourquoi SUM(quantity) et SUM(amount_millimes) répondent à des questions différentes.
- Vous distinguez le petit exemple temporaire du projet réel.

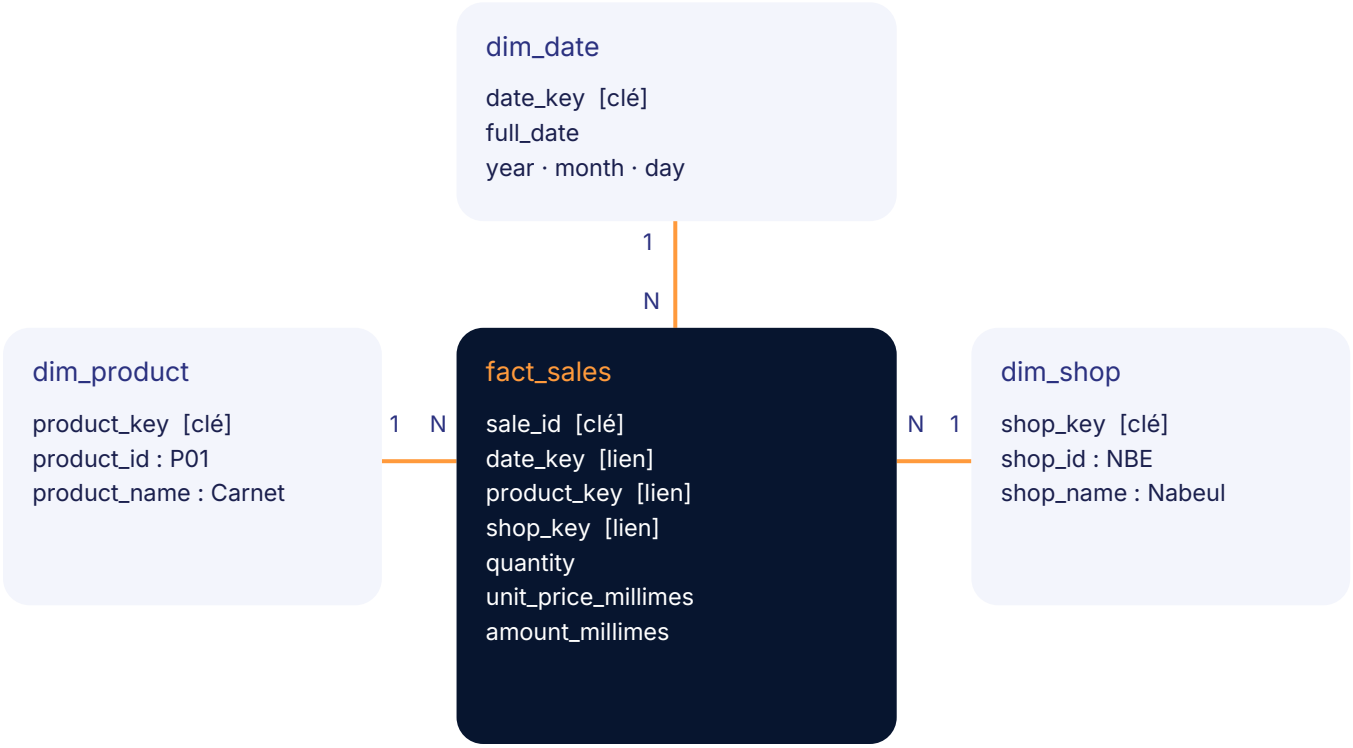
À ce stade, je sais...

Lire une requête simple, filtrer des lignes, additionner une mesure et contrôler un résultat par groupe. Nous allons maintenant organiser les tables pour rendre les questions plus claires.

MISSION 06 / DESSINER L'ENTREPÔT

06 · Pourquoi plusieurs tables ?

Le fichier de ventes contient des codes. Le catalogue explique les produits ; la liste des boutiques explique les lieux. Un entrepôt rapproche ces informations pour l'analyse. Nous allons séparer les mesures observées des descriptions qui donnent leur contexte.



1 → N : une ligne de dimension peut décrire plusieurs ventes. Chaque vente retrouve un seul produit, une seule boutique et une seule date.

La granularité, ou grain, est la phrase qui définit une ligne de la table de faits : une vente élémentaire d'un produit, dans une boutique, à une date, identifiée par sale_id. Écrivez cette phrase avant de dessiner les flèches. Mélanger ventes et totaux mensuels dans la même table rendrait les sommes ambiguës.

Un schéma en étoile

La table fact_sales est au centre. Elle référence dim_product, dim_shop et dim_date. Les dimensions décrivent les faits ; elles ne portent pas chacune une copie du montant total.

MISSION 06 / RELIER SANS MULTIPLIER

06 · Clés métier et clés de l'entrepôt

Table	Clé	Exemple de sens
dim_product	product_key	Clé interne ; product_id conserve P01.
dim_shop	shop_key	Clé interne ; shop_id conserve NBE.
dim_date	date_key	20260105 décrit le 5 janvier 2026.
fact_sales	sale_id	Identifie la vente élémentaire S001.

Le code P01 est une clé métier : il vient de la source. product_key est une clé substitut : l'entrepôt lui attribue un entier interne. Une vente utilise cette clé pour retrouver un seul produit. Les entiers attribués peuvent varier selon l'ordre de chargement : leur valeur n'est pas un classement métier.

Une clé étrangère doit pointer vers une dimension existante. Il faut donc charger les dimensions avant les faits. Une jointure relie ensuite la clé de la vente à la clé de la dimension pour retrouver, par exemple, le nom du produit.

1. Code P01 dans la source →
2. Chercher sa clé interne →
3. Placer la clé dans le fait →
4. Lire le nom par jointure

Règle de contrôle

Une vente doit retrouver exactement un produit, une boutique et une date. Une jointure qui crée plusieurs lignes pour une vente peut gonfler les montants.

MISSION 06 / ATELIER DE MODÉLISATION

06 · Construire et observer le schéma

- Sur papier, dessiner quatre rectangles : fact_sales au centre, puis les trois dimensions.
- Placer les noms des colonnes du contrat dans la bonne table.
- Relier les trois clés étrangères de fact_sales aux clés des dimensions.
- Écrire sous le dessin la phrase définissant une ligne de faits.
- Ouvrir scripts/06_modele.py, conserver TABLE_A_OBSERVER = "fact_sales", puis F5.

Le script permet d'observer la structure créée. Repérez sale_id, quantity, unit_price_millimes et amount_millimes dans les faits. Puis changez TABLE_A_OBSERVER en "dim_product", relancez et retrouvez product_key, product_id et product_name. Restaurez "fact_sales".

Une variation sur le dessin

La responsable voudrait analyser les ventes par mois. Faut-il ajouter le montant mensuel à chaque ligne de vente ? Proposez plutôt le lien avec dim_date et la colonne month. Vérifiez que la quantité garde toujours son sens initial.

Extension facultative

Une catégorie de produit pourrait devenir une colonne de dim_product. Expliquez comment ce contexte permettrait un regroupement, sans changer le grain des faits. Ne modifiez pas le schéma de référence.

MISSION 06 / PREUVE ET BILAN

06 · Vérifier le modèle avant de charger

Question	Votre contrôle
Une ligne de faits ?	Une vente élémentaire identifiée par sale_id.
Un produit peut-il avoir plusieurs ventes ?	Oui ; sa ligne de dimension est partagée.
Un fait peut-il viser P99 inconnu ?	Non ; il faut isoler ou traiter le problème explicitement.
Où va le montant de S001 ?	Dans fact_sales, avec son contexte par les clés.

La base peut être structurellement correcte et encore vide. Créer une table ne charge pas les ventes. À cette étape, nous contrôlons le plan de rangement ; la mission suivante y placera les lignes puis vérifiera les liens.

Votre preuve

- Un dessin lisible avec les quatre tables et les trois relations.
- Une phrase de granularité sans confusion avec client ou commande.
- Une explication de la différence product_id / product_key.
- Le script a été restauré sur fact_sales.

À ce stade, je sais...

Distinguer faits et dimensions, définir le grain, lire une clé métier, une clé substitut et une clé étrangère. Je sais pourquoi le chargement doit respecter un ordre.

MISSION 07 / CHARGER DANS LE BON ORDRE

07 · Du schéma vide aux ventes analysables

Nous avons des lignes propres, des montants contrôlés et un modèle. Le chargement va réunir ces éléments. Chaque produit et boutique doit d'abord exister dans sa dimension ; ensuite les faits peuvent conserver leurs clés.

1. Charger les dimensions
2. Retrouver les clés
3. Insérer les ventes
4. Contrôler et valider

Avant de valider	Résultat de référence
Dimensions produits	4 produits
Dimensions boutiques	2 boutiques
Dates des ventes initiales	3 dates
Faits	12 ventes élémentaires
Mesures	32 articles ; 99 000 millimes

Une transaction regroupe des opérations : elles sont validées ensemble ou annulées ensemble en cas d'erreur. Nous évitons ainsi une base à moitié mise à jour. Ce mécanisme ne remplace pas les règles métier ; il les aide à rester cohérentes.

Le fichier de base

L'entrepôt principal est outputs/atlas.sqlite. Ouvrir ce fichier dans le Bloc-notes n'affiche pas un tableau lisible : c'est Python et SQLite qui savent lire son format.

MISSION 07 / UN PETIT EXEMPLE

07 · Pourquoi une clé étrangère protège les liens

Exemple autonome à ouvrir dans Thonny

```
import sqlite3

base = sqlite3.connect(":memory:")
base.execute("PRAGMA foreign_keys = ON")
base.execute("CREATE TABLE produit (id TEXT PRIMARY KEY)")
base.execute("INSERT INTO produit VALUES (?)", ("P01",))
base.execute("""CREATE TABLE vente (
    produit_id TEXT REFERENCES produit(id), quantite INTEGER
)""")
base.execute("INSERT INTO vente VALUES (?, ?)", ("P01", 2))
print(base.execute("SELECT COUNT(*) FROM vente").fetchone()[0])
base.close()
```

Ouvrez examples/07_relation.py. La vente P01 est acceptée car le produit existe ; le résultat est 1. Sur une copie de cet exemple, remplacer uniquement P01 par P99 dans l'insertion de vente provoque une erreur de clé étrangère. Ce refus évite un lien vers un produit absent.

- Ouvrir scripts/07_charger.py puis F5.
- Lire les contrôles affichés et retrouver outputs/atlas.sqlite.
- Comparer les résultats aux ventes transformées, sans changer la source.

Le script 07 exécute aussi PRAGMA foreign_key_check. Le message « Liens invalides : [] » signifie qu'aucun lien vers une dimension absente n'a été trouvé. Ce contrôle complète les totaux ; il ne remplace pas la vérification du sens des données.

L'ordre a un sens

Dans le projet, une erreur ne doit pas être « réparée » en désactivant les clés étrangères. Il faut comprendre la donnée ou le chargement qui produit un lien impossible.

MISSION 07 / ATELIER DE CHARGEMENT

07 · Charger puis vérifier par une autre voie

- Écrire les quatre contrôles à vérifier avant de lancer le chargement.
- Exécuter scripts/07_charger.py et relever les résultats.
- Comparer les sommes dans le fichier transformé et dans la base.
- Expliquer comment la jointure retrouve le nom Carnet depuis une vente P01.
- Relancer le même script et observer le comportement documenté : la référence ne doit pas devenir 24 ventes ou 198.000 TND.

Un contrôle utile vient d'une autre lecture : comptez les ventes, additionnez les quantités, comparez les montants par boutique et recherchez les liens sans dimension. Une seule somme correcte ne prouve pas que chaque produit a été associé au bon nom.

Expliquez le chemin d'une vente

Dans votre journal, suivre S001 : source → ligne nettoyée → 9 000 millimes → codes produit/boutique → clés de dimension → ligne de faits. Écrivez ce qui change de forme et ce qui doit rester identique.

Extension facultative

Prédire ce qui se passerait si le chargement utilisait seulement INSERT sans contrôle de sale_id et sans clé primaire. Pourquoi une deuxième exécution pourrait-elle compter deux fois la même vente ? Ne retirez pas les protections du projet.

MISSION 07 / PREUVE ET BILAN

07 · Un entrepôt rempli n’est pas encore une preuve

Ce que vous montrez	Ce que cela établit
12 faits	Une vente distincte par identifiant retenu.
32 articles	La quantité n’a pas été perdue.
99.000 TND	Le montant initial a été conservé.
4 produits ; 2 boutiques	Les contextes du cas sont présents.
Aucun lien absent	Les clés étrangères retrouvent leurs dimensions.

Si le fichier est verrouillé, fermez l’application qui pourrait le garder ouvert puis relancez. Si une contrainte échoue, conservez le message et revenez aux données concernées. Ne supprimez pas une protection uniquement pour faire disparaître l’erreur.

Votre preuve

- Vous expliquez pourquoi les dimensions sont chargées avant les faits.
- Vous savez retrouver le fichier atlas.sqlite et citer au moins trois contrôles.
- Vous différenciez une validation technique et une justification métier.
- Votre journal suit S001 sans changer son montant.

À ce stade, je sais...

Décrire le chargement d’un schéma en étoile, contrôler les mesures et les relations, et expliquer le rôle d’une transaction. La prochaine difficulté sera d’ajouter seulement les ventes nouvelles.

MISSION 08 / CHARGEMENT INCRÉMENTAL

08 · Le lendemain, trois lignes arrivent

La responsable envoie un nouveau fichier. Tout reconstruire à partir d'un seul fichier n'est plus la seule possibilité : nous voulons ajouter les ventes absentes et reconnaître celles qui sont déjà chargées. Le fichier ventes_ajout.csv contient deux ventes nouvelles et une vente déjà connue.

Identifiant	Contenu	Décision attendue
S013	Carnet × 1 à Nabeul, le 8 janvier	Ajouter : 4.500 TND
S014	Classeur × 2 à Tunis, le 8 janvier	Ajouter : 12.000 TND
S012	Même vente que dans la base	Reconnaître : ne pas ajouter

Le montant supplémentaire est 4.500 + 12.000 = 16.500 TND. Le nouveau total est 99.000 + 16.500 = 115.500 TND. Le nombre de faits passe de 12 à 14 ; la quantité passe de 32 à 35 articles.

Ne pas utiliser seulement la date

Le fichier contient S012 avec une date plus ancienne. Dans cet atelier, nous comparons les identifiants et leurs valeurs. Une vente reçue tardivement ne doit pas être ignorée uniquement à cause de sa date.

MISSION 08 / L'IDEMPOTENCE

08 · Rejouer sans compter deux fois

Un traitement est idempotent lorsque le rejouer avec les mêmes données ne modifie plus le résultat après le premier passage réussi. C'est utile si une personne clique deux fois ou si un traitement doit être relancé après un doute.

Exemple autonome à ouvrir dans Thonny

```
deja_presents = {"S001", "S012"}
recus = ["S012", "S013", "S014"]
for identifiant in recus:
    if identifiant in deja_presents:
        print(identifiant, "deja present")
    else:
        print(identifiant, "nouveau")
        deja_presents.add(identifiant)
print(len(deja_presents))
```

Cet exemple autonome illustre la reconnaissance d'identifiants. Il ne vérifie pas le contenu des ventes. Le vrai chargement contrôle aussi les valeurs : même sale_id et mêmes valeurs = déjà présent ; même sale_id avec des valeurs différentes = conflit explicite. Nous ne remplaçons pas silencieusement l'ancienne vente.

- Ouvrir scripts/08_incremental.py, garder SOURCE = "ventes_ajout.csv".
- Vérifier que la mission 07 a déjà créé outputs/atlas.sqlite.
- Prédire le premier passage, puis appuyer sur F5.

Premier passage

Deux ventes nouvelles et une déjà présente : la base contient maintenant 14 faits et 115.500 TND.

MISSION 08 / ATELIER DE REJEU

08 · Le deuxième passage est le vrai test

- Après le premier ajout, relever le nombre de faits, d'articles et le montant.
- Prédire ce qui se passera si vous appuyez encore sur F5.
- Relancer exactement le même script sans modifier le fichier.
- Vérifier : aucune nouvelle vente, trois déjà présentes ; total inchangé.
- Expliquer pourquoi S012 ne doit augmenter ni le nombre de faits ni le montant.

État de la base	Faits	Articles	Montant
Avant l'ajout	12	32	99.000 TND
Après le premier passage	14	35	115.500 TND
Après le deuxième passage	14	35	115.500 TND

Si vous reprenez le cours plus tard, le premier passage observé peut déjà annoncer zéro nouvelle vente : les données ont peut-être été chargées lors d'une séance précédente. Vérifiez l'état de la base et votre journal avant de conclure que le script ne fonctionne pas.

Extension facultative

Sur papier, inventer une réception du même S013 avec quantité 2 au lieu de 1. Expliquer pourquoi « ignorer tout identifiant existant » masquerait une différence. Une politique de correction doit être décidée explicitement.

MISSION 08 / PREUVE ET BILAN

08 · Un journal pour expliquer ce qui s’est passé

Le suivi d’un chargement doit distinguer les lignes reçues, les lignes valides, les nouveaux faits, les faits déjà présents et les problèmes. « Trois lignes reçues » ne signifie pas « trois nouvelles ventes ». Ces comptes répondent à des questions différentes.

Situation	Comportement à expliquer
Même identifiant, mêmes valeurs	Reconnaître et conserver une seule vente.
Même identifiant, valeurs différentes	Signaler un conflit et ne pas valider partiellement.
Nouvel identifiant valide	Ajouter le contexte si nécessaire, puis la vente.
Ligne invalide	Isoler avec une raison ; ne pas inventer la valeur.

Votre preuve

- Votre journal contient les résultats du premier passage et du rejeu.
- Vous savez expliquer pourquoi 3 reçues donnent seulement 2 nouvelles ventes.
- Vous reliez les 16.500 TND ajoutés aux deux ventes réellement nouvelles.
- Vous distinguez une répétition exacte et une correction de vente.

À ce stade, je sais...

Décrire un chargement incrémental, vérifier l’idempotence et expliquer un conflit de contenu. Je sais contrôler le résultat à partir d’un état initial connu, pas seulement d’un message « terminé ».

MISSION 09 / PRÉPARER LE RAPPORT

09 · Transformer le résultat en information utile

L'entrepôt est rempli, mais la responsable ne veut pas lire du SQL. Nous allons produire un rapport local qui indique la période, les unités, les volumes et les montants par produit et par boutique. Il doit aussi rappeler les limites du jeu de données.

1. Entrepôt contrôlé
2. Requêtes de synthèse
3. Rapport HTML local
4. Conclusion justifiée

Indicateur après l'ajout	Valeur	Sens
Ventes élémentaires	14	Nombre de faits distincts.
Articles	35	Somme des quantités.
Montant	115.500 TND	Somme des ventes observées.
Période	5 au 8 janvier 2026	Dates des ventes chargées.

Un graphique compare des valeurs. Son titre doit dire la question et son axe doit indiquer l'unité. Une barre plus longue en TND ne signifie pas forcément davantage d'articles. Un produit cher peut générer plus de montant avec moins d'unités.

Une décision prudente

Le rapport peut attirer l'attention sur un produit à examiner. Pour décider d'un réapprovisionnement, il manque notamment le stock disponible et les délais fournisseurs.

MISSION 09 / DU SCRIPT AU NAVIGATEUR

09 · Générer et retrouver le bon fichier

- Ouvrir scripts/09_rapport.py dans Thonny.
- Vérifier que la mission 08 a été réalisée et que la base contient 14 faits.
- Garder TITRE_RAPPORT = "Atlas Papeterie — ventes observées".
- Appuyer sur F5. Dans l'Explorateur, ouvrir outputs/rapport_atlas.html avec votre navigateur.
- Lire le total, les dates et les regroupements. Le fichier reste sur votre ordinateur.

Le rapport HTML est une page consultable localement. Le script le reconstruit depuis la base : modifier le texte du rapport à la main ne corrige pas les données. Après une nouvelle exécution, utilisez Ctrl+R dans le navigateur pour afficher la version régénérée.

Produit après l'ajout	Montant
Carnet	27.000 TND
Stylo	18.000 TND
Classeur	48.000 TND
Surligneur	22.500 TND
Total	115.500 TND

Vérifier au lieu de supposer

Avant de commenter une barre, retrouvez sa valeur et sa période. Si le total affiche encore 99.000 TND, vérifiez l'état de l'entrepôt et l'exécution de la mission 08.

MISSION 09 / ATELIER DE COMMUNICATION

09 · Une conclusion en trois phrases

Votre travail ne s'arrête pas à produire le rapport. Il faut expliquer une observation, la relier aux données puis dire ce qui manque pour décider. Utilisez des phrases courtes, avec les unités et la période.

- Observation : choisir un produit et citer son montant sur la période du rapport.
- Vérification : montrer la requête ou le tableau qui soutient cette valeur.
- Limite : nommer une information absente qui pourrait changer la décision.

Une variation visible

Modifier seulement TITRE_RAPPORT pour ajouter votre prénom ou votre groupe, enregistrer et relancer. Actualiser le navigateur. Le titre doit changer ; le total et les regroupements doivent rester identiques. Restaurer ensuite le titre de référence et régénérer le rapport.

Une question que vous choisissez

Choisissez une question parmi : quel produit représente le plus de montant ? Quelle boutique a le plus de ventes observées ? Pourquoi les unités vendues et les montants peuvent-ils raconter des choses différentes ? Répondez avec une valeur, son unité et une réserve.

Extension facultative

Les CSV de synthèse peuvent ensuite être importés dans un outil BI. Cette extension n'exige ni compte ni licence pour réussir le socle de cet atelier ; le rapport local suffit.

MISSION 09 / BILAN ET PRÉPARATION DU DÉFI

09 · La preuve est dans le chemin complet

Une présentation solide permet de remonter du rapport à la source. Pour un nombre choisi, montrez les lignes qui l'alimentent, la transformation appliquée et la requête qui l'additionne. Vous n'avez pas besoin de réciter tout le moteur technique.

À préparer pour la séance finale

- Conserver un dossier du cours complet avec les scripts et les résultats.
- Rédiger une conclusion de trois phrases dans votre journal.
- Choisir une vente et expliquer son trajet fichier → contrôle → entrepôt → rapport.
- Préparer une question sur une erreur ou un choix que vous avez dû comprendre.
- Laisser les paramètres de référence restaurés.

La séance finale dure une heure. Elle utilise une base séparée pour réparer une petite donnée, rejouer le traitement et présenter vos contrôles. La synthèse du projet doit donc être préparée maintenant, pas improvisée à la dernière minute.

À ce stade, je sais...

Produire un rapport, vérifier ses unités et sa période, expliquer un chiffre avec sa source et formuler une conclusion limitée à ce que les données permettent.

MISSION 10 / DÉFI FINAL · 1 HEURE

10 · Un défaut à réparer, une preuve à montrer

Vous allez rejouer le pipeline dans un espace séparé, puis réparer une vente invalide. Le but est de démontrer une méthode : prévoir, corriger la bonne copie, contrôler le changement et rejouer sans doubler les ventes.

- Ouvrir scripts/10_defi.py dans Thonny.
- Garder SOURCE_AJOUT = DATA / "ventes_ajout.csv" pour le premier passage.
- Exécuter avec F5. Le script construit outputs/10/defi.sqlite.
- Ouvrir outputs/10/rapport_defi.html : retrouver 14 ventes, 35 articles et 115.500 TND.
- Repérer la copie préparée outputs/10/ventes_a_reparer.csv.

Le défi travaille dans outputs/10/. La base principale outputs/atlas.sqlite et son rapport restent séparés. Ne changez pas data/ventes_initiales.csv ni data/ventes_ajout.csv. Le défi copie un fichier d'exercice à réparer ; ce fichier n'est pas une correction autorisée de vraies données métier.

Votre départ

La référence du défi doit être connue avant la réparation. Si vous avez déjà réalisé ce défi, son état peut être plus avancé : relevez-le dans votre journal et demandez un nouveau dossier extrait si nécessaire.

MISSION 10 / UNE DONNÉE, UNE RAISON

10 · Prévoir le changement avant la correction

Ouvrez outputs/10/ventes_a_reparer.csv dans Thonny ou le Bloc-notes. Trouvez S015. Sa quantité nulle explique son rejet. L'énoncé confirme que la quantité correcte pour l'exercice est 2. C'est cette information fournie qui autorise la correction ; nous ne choisissons pas une valeur au hasard.

Vente à réparer	Valeur à utiliser
sale_id	S015
Quantité corrigée	2 articles
Prix unitaire	1.500 TND
Montant à ajouter	À calculer avant l'exécution

- Calculer le montant de S015 en millimes et en TND.
- Prévoir le nouveau nombre de faits et d'articles.
- Écrire vos trois prédictions dans le journal.
- Changer uniquement la quantité de S015 dans la copie à réparer ; enregistrer.
- Dans le script 10, choisir SOURCE_AJOUT = DOSSIER / "ventes_a_reparer.csv".

Un indice si vous hésitez

Le point de départ contient déjà 14 faits et 35 articles. Une vente nouvelle de deux articles ajoute un fait, pas deux. Le montant s'ajoute une seule fois.

MISSION 10 / PRATIQUE AUTONOME

10 · Rejouer, comparer, expliquer

- Exécuter le script 10 après la réparation et relever les contrôles.
- Ouvrir le rapport du défi et actualiser le navigateur.
- Comparer vos prédictions aux résultats réellement produits.
- Exécuter une deuxième fois avec le même fichier réparé.
- Vérifier que le total ne change plus ; expliquer pourquoi.

Votre courte démonstration

Montrez le fichier corrigé, le résultat du chargement et le rapport. Expliquez la règle qui provoquait le rejet, la raison de la nouvelle valeur et le contrôle qui montre que le rejeu ne crée pas une deuxième vente. Terminez par une limite du rapport pour une décision d'achat.

Vous pouvez présenter à deux, mais chacun doit expliquer une partie différente et répondre à une question simple. Une explication écrite courte peut remplacer la prise de parole si elle permet de montrer la même compréhension.

Si le total ne change pas

Vérifiez que vous avez corrigé outputs/10/ventes_a_reparer.csv et choisi cette copie dans SOURCE_AJOUT. Vérifiez aussi que vous avez enregistré les deux fichiers avant F5. Ne modifiez pas le moteur pour forcer le résultat.

MISSION 10 / PREUVE ET BILAN FINAL

10 · Rendre le projet compréhensible à quelqu'un d'autre

Votre dossier de preuves

- Les scripts et données sont conservés avec leur arborescence.
- Votre journal montre une prédiction, un essai, une correction et un rejeu.
- Le rapport du défi est ouvrable depuis outputs/10/.
- Votre dessin distingue faits, dimensions et grain.
- Votre conclusion cite une période, une unité et une limite.

Je peux expliquer…	Sans réciter le code complet
E · Extract	D'où viennent les données et ce que représente une ligne.
T · Transform	Les règles, les rejets et le calcul des montants.
L · Load	L'ordre des tables, les clés et le contrôle du rejeu.
BI	Le chiffre retenu, sa source et les limites de la décision.

Votre progression se voit dans vos essais et vos explications. Un résultat copié sans compréhension est fragile ; une erreur comprise et réparée est une preuve d'apprentissage. Gardez le projet pour le réutiliser sur un autre petit jeu de données.

Après le défi

Remettre SOURCE_AJOUT sur ventes_ajout.csv ne retire pas S015 déjà chargée : le traitement ajoute et reconnaît, il ne supprime pas. Pour recommencer au départ, extraire une nouvelle copie complète du cours dans un nouveau dossier. Conserver l'ancien dossier comme preuve.

À ce stade, je sais…

Rejouer un petit pipeline ETL, réparer une donnée avec une justification, préserver la source, contrôler les résultats et expliquer un projet BI depuis le fichier jusqu'au rapport.

SYNTHÈSE / CE QUE VOUS AVEZ CONSTRUIT

Le projet entier, en une lecture

1 16 lignes reçues → 2 12 ventes initiales → 3 14 après ajout → 4 Rapport contrôlé

Au départ, les ventes étaient du texte dans un fichier. Vous avez appris à retrouver les colonnes, normaliser des codes, isoler des anomalies et calculer des montants. Puis vous avez organisé les mesures dans un schéma en étoile, chargé les tables et ajouté de nouvelles ventes sans doubler celles déjà connues.

Étape	Preuve utile
Comprendre	Phrase de granularité et dictionnaire des colonnes.
Contrôler	12 retenues + 1 doublon + 3 rejets = 16 reçues.
Transformer	S001 vaut 9 000 millimes ; total initial 99.000 TND.
Charger	14 faits et 115.500 TND après l'ajout.
Expliquer	Rapport, conclusion et limite métier.

Le défi S015 utilise un espace séparé. Il ne change pas la référence de la base principale. Garder les deux états distincts vous aide à expliquer exactement quel fichier, quelle base et quelle version du rapport vous présentez.

Une habitude à conserver

Avant d'exécuter, prévoir. Après l'exécution, comparer. Avant de conclure, retrouver la source et dire ce qui manque.

POUR CONTINUER / UNE EXTENSION À LA FOIS

Après ces 28 heures

Vous avez construit un premier pipeline local, sur un jeu volontairement petit. Ce n'est pas encore un système de production. Le prochain progrès consiste à garder la méthode tout en augmentant une seule difficulté à la fois.

- Nouveau fichier : adapter les noms de colonnes et documenter le nouveau grain.
- Plus de transformations : découvrir pandas après avoir compris les opérations réalisées.
- Nouvel outil BI : importer des synthèses dans Power BI si l'environnement le permet.
- Plus de fiabilité : enrichir les contrôles et examiner les journaux.
- Historique : étudier ce qui arrive lorsqu'un produit change de catégorie ; introduction aux dimensions à évolution lente.

Votre prochaine expérience

Choisissez un petit fichier public ou fictif ne contenant aucune donnée personnelle. Écrivez d'abord une question, le sens d'une ligne et les unités. Listez trois anomalies possibles. Construisez un exemple de quelques lignes avant d'augmenter le volume.

Votre boussole

L'outil peut changer. La méthode reste : comprendre la source, rendre les règles visibles, contrôler les liens et les totaux, puis expliquer une information utile avec ses limites.

Pour retrouver une commande ou une erreur : 03_reperes_et_depannage.pdf. Pour confronter une tentative à une autre solution : 04_indices_et_corriges.pdf. Les sources officielles des outils sont rappelées dans le guide de démarrage.